

Securitize EVM DSToken ScaledUI Amount Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Stalin

ArmorMadeOfWoe

October 21, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**

5 Audit Scope **3**

6 Executive Summary **3**
- 6.1 Centralization Risks . . . 3
- 6.2 Operational Notes . . . 3
  - 6.2.1 Switching `SCALED_UI` to `REBASING` after a split is multi-step and changes what pending amounts mean . . . 3
  - 6.2.2 In `SCALED_UI`, transfer amounts are raw: UI amounts must be converted with `fromUIAmount` at execution . . . 4
  - 6.2.3 Upgrading an existing DSToken proxy keeps it in REBASING mode; ERC-8056 must be enabled by MASTER afterwards . . . 5
  - 6.2.4 Requirements to switch a token between REBASING and SCALED_UI mode . . . 5
- 6.3 Storage Layout Upgrade Safety . . . 7
- 6.4 Protocol Invariants . . . 7
  - 6.4.1 DSToken - share and raw supply conservation . . . 7
  - 6.4.2 DSToken - SCALED_UI display and rounding . . . 9
  - 6.4.3 TokenLibrary - UI multiplier schedule . . . 9
  - 6.4.4 ERC-8056 and ERC-165 interface . . . 10
  - 6.4.5 Mode switching . . . 11
  - 6.4.6 SecuritizeRebasingProvider . . . 11
  - 6.4.7 Mint throttle and over-cap mints . . . 12
  - 6.4.8 ServiceConsumer and access . . . 13
  - 6.4.9 Storage and upgrade . . . 13

7 Findings **16**
- 7.1 Low Risk . . . 16
  - 7.1.1 `DSToken::setScaledUIAmountMode` enables SCALED_UI after a non-unit rebasing history, so for tokens with fewer than 18 decimals `balanceOf` stays nearest-rounded and `sum(balanceOfUI)` can exceed `totalSupplyUI` . . . 16
- 7.2 Informational . . . 19
  - 7.2.1 `DSToken` never checks that its rebasing provider is frozen for it when SCALED_UI is enabled or the provider is re-pointed, so a later provider multiplier change rescales every raw balance 19
  - 7.2.2 `SecuritizeRebasingProvider::setMultiplier` reverts whenever `DS_TOKEN` points at a token without `scaledUIAmountEnabled`, so wiring the provider before the token is upgraded blocks every rebase of a REBASING token . . . 21
  - 7.2.3 `TokenLibrary::setUIMultiplier` accepts any non-zero multiplier with no magnitude or ratio bound, so a mis-scaled value floors the ERC-8056 views to zero or makes them revert once it applies . . . 22
  - 7.2.4 Any ISSUER can make a queued master-timelock `DSToken::setScaledUIAmountMode(false)` revert at execution by leaving a UI multiplier update pending, even one that changes no displayed value . . . 23
  - 7.2.5 Pending over-cap mints scheduled in `SCALED_UI` mode keep their raw amount across a switch to Rebase mode, so a split re-applied through the rebasing provider before execution dilutes the recipient . . . 24

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

`DSToken` is Securitize's ERC-20 security token for tokenized real-world assets. Every transfer, issuance, burn and seizure goes through on-chain services the token is wired to: a compliance service that validates each movement against investor and jurisdiction rules, a registry of verified investors, a lock manager, and a trust service that grants the `MASTER`, `ISSUER` and `TRANSFER_AGENT` roles. Balances are stored internally as 18-decimal shares and converted to token amounts through a rebasing provider, which lets MASTER apply corporate actions such as stock splits by changing a single multiplier. Issuance is throttled by a mint cap per time window, and mints above it go through a timelock that can be cancelled before they execute.

The audited change adds `ERC-165` interface detection and a second operating mode based on `ERC-8056` (Scaled UI Amount). In the new `SCALED_UI` mode, which is the default for newly deployed tokens, `balanceOf`, `totalSupply`, transfers and events stay in raw units. Corporate actions are expressed instead as a display multiplier that `ERC-8056`-aware wallets and integrators apply through `uiMultiplier`, `balanceOfUI`, `totalSupplyUI` and the `toUIAmount` / `fromUIAmount` conversions, which round down. `ISSUER` schedules a new display multiplier with an effective time; the change takes effect automatically at that time, and it can be superseded or cancelled while it is still pending. Compliance, locks and the mint cap keep working on raw amounts and never read the display multiplier.

The existing `REBASING` mode is kept, and proxies upgraded from earlier versions stay in it. `MASTER` can switch a token between the two modes only when no number an integrator reads would change: entering `SCALED_UI` requires the rebasing multiplier to be exactly 1e18, and leaving it requires the display multiplier to be 1e18 with no update pending. While a token is in `SCALED_UI`, the rebasing provider refuses to change its multiplier, so balances are never scaled twice; this lock applies once the provider is linked back to the token. The `ERC-8056` interfaces are advertised through `ERC-165`, and the `ERC-8056` functions are callable, only in `SCALED_UI` mode.

To keep `DSToken` within the EIP-170 contract size limit, the change also moves the over-cap mint timelock logic from `DSToken` into `TokenLibrary`, and routes the role-check modifiers in `ServiceConsumer` through internal functions, with the same checks and revert messages.

5 Audit Scope

The audit scope was limited to the following files changed in PR114:

```
contracts/token/IScaledUIAmount.sol
contracts/token/DSToken.sol
contracts/token/TokenLibrary.sol
contracts/data-stores/TokenDataStore.sol
contracts/rebasing/SecuritizeRebasingProvider.sol
contracts/service/ServiceConsumer.sol
```

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the EVM DSToken ScaledUI Amount code provided by Securitize. The findings consist of 1 Low severity issue with the remainder being informational.

6.1 Centralization Risks

The protocol by its regulated nature is highly centralized, with protocol admins having complete power over the token, including the ability to upgrade DSToken, `TokenLibrary`, the rebasing provider and the other DS services to new implementations; protocol users must place complete trust in the protocol admins when engaging with the protocol. One benefit to this is that even if a serious bug should be discovered once the protocol is live, protocol admins will in many cases be able to upgrade the relevant contracts and resolve the issue.

The changes in this audit add two admin powers over what holders and integrators see. - `MASTER` switches a token between `REBASING` and `SCALED_UI` mode and, in `REBASING` mode, changes the rebasing multiplier, which rescales every holder's `balanceOf` at once. `MASTER` also chooses which rebasing provider each token uses, and the lock that keeps that multiplier at `1e18` in `SCALED_UI` mode relies on `MASTER` keeping the token and its provider linked correctly. After the governance handover, `MASTER` is a timelock with a suggested 48h delay, so these actions are visible on-chain before they execute.

- `ISSUER` sets the `ERC-8056` display multiplier that integrators use to value positions in `SCALED_UI` mode. This is deliberately not timelocked: an update can be scheduled to take effect one second later, and only `ISSUER` or `MASTER` can cancel it before then. `ISSUER` also issues tokens, which the mint cap and the timelock for mints above the cap limit, while `ISSUER` and `TRANSFER_AGENT` can burn, and `TRANSFER_AGENT` can seize, holders' tokens instantly, as part of the regulated emergency toolbox.

6.2 Operational Notes

6.2.1 Switching `SCALED_UI` to `REBASING` after a split is multi-step and changes what pending amounts mean

Switching from ScaluedUI mode to Rebasing is supported, but only when the UI multiplier is exactly `1e18`, is in effect, and has nothing pending. If the token never split, the switch changes no value. If the token already reflected a split through `uiMultiplier`, there is no direct migration. The split has to be unwound on the display layer and re-applied through the rebasing provider:

1. ISSUER: `setUIMultiplier(1e18, effectiveAt)`. Once `effectiveAt` passes, `balanceOfUI` falls back to pre-split values.
2. MASTER (timelocked): `setScaledUIAmountMode(false)`. No value changes.
3. MASTER (timelocked): `provider.setMultiplier(splitMultiplier)`, where `splitMultiplier` is the split being re-applied (e.g. `2e18`). `balanceOf` of every holder is multiplied by that split.

These steps cannot be atomic. `setUIMultiplier` requires `effectiveAt > block.timestamp`, and step 2 reverts until `effectiveAt` has passed. Between that `effectiveAt` and the execution of steps 2–3, ERC-8056 integrators therefore see positions valued at the pre-split amount. The runbook documents only the switch guards, not this sequence.

The switch also changes the unit of every amount argument (`issueTokens*`, `burn`, `seize`, `transfer*`, over-cap mints). It goes from raw, pre-split units to post-split tokens once step 3 lands. Over-cap mints are the case that persists in storage: `scheduleOverCapIssuance` stores `amount` as passed, and `executeOverCapMint` converts it at the multiplier in force at execution. For example, an over-cap mint for 100 stocks is scheduled in `SCALED_UI` at 1:1 (`amount = 100e6`). The token then switches and is rebased 2:1 before execution. Alice receives 100 post-split stocks, half of her entitlement. The same applies to a REBASING token rebased while a mint is pending. That behaviour predates this PR, since the over-cap flow was moved unchanged into `TokenLibrary`. In `SCALED_UI`, a split during the wait is applied implicitly, because the stored raw amount is pre-split. Related finding: Pending over-cap mints scheduled in `SCALED_UI` mode keep their raw amount across a switch to Rebase mode, so a split re-applied through the rebasing provider before execution dilutes the recipient.

6.2.2 In `SCALED_UI`, transfer amounts are raw: UI amounts must be converted with `fromUIAmount` at execution

In `SCALED_UI` mode, `transfer` / `transferFrom` take raw, pre-corporate-action amounts. A split or consolidation changes only the ERC-8056 display (`uiMultiplier`, `balanceOfUI`). ERC-8056-aware front ends must therefore convert a UI amount entered by the user with `fromUIAmount` before calling `transfer`. Wallets and custodians that are not ERC-8056-aware show the raw `balanceOf`, which is the pre-split balance. Passing a UI number unconverted does not revert: it sends `uiMultiplier` times the intended amount.

Example. Take a 6-decimal token. Alice is issued 1 stock (`issueTokens(alice, 1e6)`), and a 2:1 split is then scheduled with `setUIMultiplier(2e18, effectiveAt)`. Once `effectiveAt` has passed, Alice's position is:

	`balanceOf` (raw)	`balanceOfUI` (displayed)
Alice	1,000,000 (1.0 token in an ERC-20 wallet)	2,000,000 (2 stocks)

Alice wants to send 1 of her 2 stocks to Bob.

Correct. The front end converts the typed UI amount at the multiplier currently in effect:

```
raw = fromUIAmount(1_000_000)          // floor(1e6 × 1e18 / 2e18) = 500_000
transfer(bob, 500_000)
```

After	`balanceOf`	`balanceOfUI`	`Transfer` event
Alice	500,000	1,000,000 (1 stock)	`Transfer(alice, bob, 500000)`
Bob	500,000	1,000,000 (1 stock)	

Wrong. The front end, or the holder in a plain ERC-20 wallet, passes the stock count as the amount:

```
transfer(bob, 1_000_000)               // "1 stock", not converted
```

After	`balanceOf`	`balanceOfUI`	`Transfer` event
Alice	0	0	`Transfer(alice, bob, 1000000)`
Bob	1,000,000	2,000,000 (2 stocks)	

The call passes every check, because `1e6 <= balanceOf(alice)`. Alice has sent her whole position, twice what she intended. A plain ERC-20 wallet makes this easy to do: it shows Alice a balance of 1.0, so typing "1" looks like

sending her one stock. An amount above the raw balance (for example `2_000_000`) reverts on the balance check, so the failure is silent only while the unconverted amount fits in the raw balance.

The conversion has to use the multiplier in effect when the transaction executes. `uiMultiplier()` switches at `block.timestamp >= effectiveAt`. A front end that computes `fromUIAmount` just before `effectiveAt` and lands the transaction just after it gets the same 2× error: it computes `1_000_000` at 1e18, which executes as 2 stocks at 2e18.

Full exits and dust. `fromUIAmount` and `toUIAmount` both floor. A holder who sends "everything shown" with `transfer(fromUIAmount(balanceOfUI(holder)))` can therefore leave raw dust behind when the ratio is not exact. For example, with a 1.5 to 1 reverse split (`uiMultiplier = 666666666666666666`) on a 6-decimal token:

Step	Raw	`balanceOfUI`
Alice holds	1,500,000	999,999 (0.999999 stock)
Alice sends `fromUIAmount(999,999)` = 1,499,998	2 left	1 (0.000001 stock)
UI multiplier set back to 1e18	2 left	2 (0.000002 stock)

At small multipliers, dust can display as zero: at `666666666666666666`, `toUIAmount(1) = 0`. The dust can always be moved, because transfers are raw. But `ComplianceServiceRegulated` lowers the investor count only when a transfer equals the holder's whole `balanceOfInvestor`. A holder who exits "as displayed" therefore stays counted as an investor (and stays in the token's wallet list), which matters for investor-count limits such as the maximum number of US or accredited investors.

The same flooring affects issuance at inexact ratios. `issueTokens(fromUIAmount(1 stock))` shows 0.999999 stock at both `666666666666666666` and `1.5e18`. The smallest amount that shows 1.000000 (1,500,001 raw at the reverse split) over-issues by one raw unit, which shows as 1.500001 stock once the multiplier is set back to 1e18.

6.2.3 Upgrading an existing DSToken proxy keeps it in REBASING mode; ERC-8056 must be enabled by MASTER afterwards

An existing proxy can be upgraded to the new implementation, but the upgrade on its own does not turn on ERC-8056. After the upgrade the token stays in **REBASING** mode until MASTER switches it explicitly.

Why the token stays in REBASING. The new state goes in four slots taken from `__gap` (`scaledUIAmountMode` plus `scaledUIAmount {multiplier, newMultiplier, effectiveAt}`; gap 28 to 24). On an upgraded proxy these slots read as zero, and `scaledUIAmountMode == false` means REBASING. **New tokens start in** `SCALED_UI` **only because** `DSToken.initialize` **sets the flag**. An existing proxy has already run its initializer and never runs it again. After the upgrade:

- `balanceOf`, `totalSupply` and transfers behave exactly as before, scaled by the rebasing provider.
- `setMultiplier` on the provider keeps working.
- Every ERC-8056 function, including `toUIAmount` / `fromUIAmount`, reverts with `Scaled UI amount not enabled`.

6.2.4 Requirements to switch a token between REBASING and SCALED_UI mode

A DSToken runs in either REBASING or SCALED_UI (ERC-8056) mode. The only way to switch is:

```
DSToken.setScaledUIAmountMode(bool enabled)   // MASTER, or owner(); timelocked after handover
```

The on-chain guards ensure the switch does not change any number an integrator reads. They allow it only while both multipliers are at identity (`1e18`). The table below lists what each direction needs.

	REBASING to SCALED_UI (`true`)	**SCALED_UI to REBASING** (`false`)
Caller	MASTER, or `owner()`	MASTER, or `owner()`
Mode	Token must currently be REBASING, else `Scaled UI amount mode unchanged`	Token must currently be SCALED_UI, else `Scaled UI amount mode unchanged`
Multiplier guard	`token.getDSService(REBASING_PROVIDER).multiplier()` must be exactly `1e18`, else `Rebasing multiplier must be 1e18`	UI multiplier in effect must be `1e18` with nothing pending: `newUIMultiplier() == 1e18` and `block.timestamp >= effectiveAt()`, else `UI multiplier must be 1e18 with no pending update`
Wiring	`REBASING_PROVIDER` must be set on the token. If it is unset, the call to `multiplier()` on `address(0)` reverts	No wiring is checked
State written	UI data reset to identity (`multiplier = newMultiplier = 1e18`, `effectiveAt = 0`)	UI data left as is. It is not read in REBASING mode and is reset on the next enable
Events	`UIMultiplierUpdated(0, 1e18, now)`, `ScaledUIAmountModeChanged(true)`	`ScaledUIAmountModeChanged(false)`
After the switch	ERC-8056 getters live. `supportsInterface` reports the four ERC-8056 ids. `provider.setMultiplier` reverts, but only if the provider has this token under `DS_TOKEN` (ON-1)	ERC-8056 getters revert. `supportsInterface` drops the ERC-8056 ids. `provider.setMultiplier` works again

Holders may exist at either switch. With both multipliers at `1e18`, `balanceOf` and `balanceOfUI` already agree, so no balance changes.

What the guards do not cover. The operator has to handle these:

1. **A token that has already rebased cannot move to SCALED_UI.** If the provider multiplier is not `1e18`, the enable guard reverts. Resetting the multiplier to `1e18` first changes every holder's `balanceOf`, and the contracts have no share-migration path. In practice, only tokens that never rebased, or whose multiplier is exactly back at `1e18`, can be enabled.
2. **A token that has split in SCALED_UI needs an unwind.** The UI multiplier must first be scheduled back to `1e18`, and the disable can execute only after that `effectiveAt`.
3. **A pending update blocks the disable even when it targets `1e18`.** An update scheduled back to `1e18` still makes `setScaledUIAmountMode(false)` revert until `effectiveAt` passes. Cancelling (`cancelUIMultiplierUpdate`) restores the previous multiplier, so cancelling helps only if that multiplier is `1e18`. Because the switch is timelocked, its execution time must fall after that `effectiveAt`, and nobody may schedule another update in between.
4. **The enable checks only the provider wired at that moment.** A later `DSToken.setDSService(REBASING_PROVIDER, ...)`, `provider.setDSService(DS_TOKEN, ...)` or a missing `DS_TOKEN` back-link can break the `1e18` invariant after the switch.
5. **The units in off-chain tooling change.** In `SCALED_UI`, amount arguments are raw. In `REBASING` they are post-rebase tokens. At identity the two are the same, but operator tooling and integrators must change

convention at the switch. Tell ERC-8056 integrators in advance, because the ERC-165 answers and the getters' availability change in the same transaction.

6. **Upgraded proxies start in REBASING** and need explicit switch to `SCALED_UI` mode afterwards.

6.3 Storage Layout Upgrade Safety

Securitize asked for an explicit opinion on whether existing DSToken proxies can be upgraded to the audited commit (`29690b11`) without corrupting storage. We compared the compiler-generated storage layout of the audited DSToken against the layout of the rebasing-era release from the repository history: at the unreleased base of this change (`fb8a3b59`). Which uses OpenZeppelin 5, which keeps its own state (initialization, UUPS, EIP-712 and nonces) in fixed namespaced slots outside the token's layout.

Proxies already running a rebasing-era implementation can be upgraded safely. Every variable up to slot 65 keeps its position, size and type in all releases, and every variable added since then (the mint cap and over-cap state, and the four new SCALED_UI slots) sits in space that was previously part of the storage gap, which still ends at slot 101. On an upgraded proxy these slots read as zero, which means the mint cap is disabled, no over-cap mints are pending, and the token stays in REBASING mode with the ERC-8056 functions disabled until MASTER enables SCALED_UI. In this change the `PendingMint` struct moved from `TokenDataStore` to `TokenLibrary` with identical fields and offsets. We also performed a real upgrade of a proxy deployed from the unreleased base of this change on a local node: the OpenZeppelin upgrade storage check passed, every balance, setting and pending over-cap mint was preserved, the pending mint executed correctly afterwards, and the upgraded token could then be switched to `SCALED_UI` with the rebasing multiplier locked.

Proxies deployed before rebasing was introduced (before BC-1257, `04a329e`) should not be upgraded directly. Their storage slots line up, because the rebasing change added no token storage, but their balances are stored in token units while the audited code reads them as 18-decimal shares; for a 6-decimal token every balance would read 10^12 times too small, and `balanceOf` reverts until a rebasing provider is wired. Tokens on Securitize's legacy (non-ERC-1967) proxy additionally need a proxy migration, and their source is not in this repository. Upgrading these proxies requires a balance migration outside the scope of this change.

Two related points apply to upgrades from older releases, although neither is introduced by this change. The permit (EIP-712) support was added after `4.0.0` without a `reinitializer`, so a `3.1.5` or `4.0.0` proxy upgraded straight to the audited commit keeps an empty EIP-712 version, and permit signatures created with version "1" will not verify. In addition, a safe upgrade also depends on linking the new `TokenLibrary`, upgrading the token before the rebasing provider, and wiring the provider back to the token before SCALED_UI is enabled.

6.4 Protocol Invariants

The following potential invariants were identified:

6.4.1 DSToken - share and raw supply conservation

ID	Property
INV-01	The sum of `tokenData.walletsBalances` over all wallets equals `tokenData.totalSupply` (both in shares) after every `transfer`, `transferFrom`, `transferWithPermit`, `issueTokens`, `issueTokensCustom`, `issueTokensWithMultipleLocks`, `executeOverCapMint`, `burn` and `seize`, in either mode
INV-02	Each issuance (throttled or over-cap) raises `tokenData.totalSupply`, `tokenData.totalIssued` and the recipient's `tokenData.walletsBalances` entry by the same share amount, each `burn` lowers `tokenData.totalSupply` and the burned wallet's entry by the same share amount, and transfers and seizures change neither `tokenData.totalSupply` nor `tokenData.totalIssued`
INV-03	`tokenData.totalIssued` never decreases and `tokenData.totalSupply <= tokenData.totalIssued` (shares) in every reachable state

ID	Property
INV-04	While SCALED_UI, the sum of `balanceOf(w)` over all wallets equals `totalSupply()` (raw) in every reachable state, including on a token that entered SCALED_UI from REBASING after rebasing history
INV-05	While SCALED_UI, a successful `transfer`, `transferFrom`, `transferWithPermit` or `seize` of raw `v` from `a` to `b != a` lowers `balanceOf(a)` by exactly `v`, raises `balanceOf(b)` by exactly `v` and leaves `totalSupply()` unchanged
INV-06	While SCALED_UI, a successful issuance of raw `v` (any `issueTokens*` entry point, or `executeOverCapMint` of an operation scheduled with `amount` `v`) raises the recipient's `balanceOf` and `totalSupply()` by exactly `v`, and a successful `burn` of raw `v` lowers both by exactly `v`
INV-76	A successful `issueTokensWithMultipleLocks` with equal-length `_valuesLocked` and `_releaseTimes` arrays to a wallet registered to a non-empty investor id records, for that investor id, one manual lock per submitted pair with the submitted amount, reason and release time, and a successful `issueTokensCustom` to such a wallet records one such lock when its `_valueLocked` is non-zero and none otherwise; issuance to a wallet with no registered investor id records no manual lock for any investor id, the empty id included; in either mode issuance rejects arrays of unequal length and submitted lock amounts that sum to more than the issued amount argument `_value`
INV-07	While SCALED_UI, on a token whose every wallet share balance is a multiple of `10 ** (18 - decimals)` (every 18-decimal token, and every token whose provider multiplier has been `1e18` since its first issuance), every holder `a` can move or have removed its full `balanceOf(a)`: `transfer(to, balanceOf(a))` by `a`, `burn(a, balanceOf(a), reason)` and `seize(a, to, balanceOf(a), reason)` are never rejected by the share-balance check
INV-08	While SCALED_UI, on a token whose provider multiplier has been `1e18` since its first issuance and where no wallet has been reassigned to a different investor in the registry while holding a non-zero share balance, for every non-empty registered investor id `id`, `tokenData.investorsBalances[id]` (shares) equals the sum of `tokenData.walletsBalances` over the wallets registered to `id`, and `balanceOfInvestor(id)` equals the sum of `balanceOf` over those wallets (raw); on every SCALED_UI token, each successful `transfer`, `transferFrom`, `transferWithPermit`, issuance, `burn` or `seize` changes `tokenData.investorsBalances[id]` by exactly the net share change of the wallets registered to `id` whenever that entry is a multiple of `10 ** (18 - decimals)` before the operation
INV-09	No UI-multiplier event (`setUIMultiplier`, `cancelUIMultiplierUpdate`, a mode switch, or time reaching `effectiveAt()`) changes any wallet or investor share balance, `balanceOf`, `totalSupply()`, allowance, lock, `mintedInWindow`, `windowStart` or `pendingMints` record
INV-10	While SCALED_UI, every token-side share conversion is exact identity on raw amounts in every reachable state, including after any `setDSService` rewiring of `REBASING_PROVIDER` or `DS_TOKEN`: raw `x` converts to exactly `x * 10 ** (18 - decimals)` shares, and `balanceOf`, `totalSupply()`, `balanceOfInvestor` and `totalIssued` return the stored shares divided by `10 ** (18 - decimals)` with no remainder
INV-11	While SCALED_UI, every amount argument of an ERC-20 function (`transfer`, `transferFrom`, `approve`, `permit`) and every value `allowance` returns, every amount passed to compliance (`validateTransfer`, `validateIssuance`, `validateBurn`, `validateSeize`, `preTransferCheck`) and every amount passed to investor locks is raw (the raw amount of the operation, and for the balance argument of `validateTransfer` the sender's raw `balanceOf`), and the mint cap and share-denominated lock and issuance records meter the shares that raw amount converts to, each independent of `uiMultiplier`
INV-12	While SCALED_UI, every `Transfer` event's value equals the raw amount of the operation it records, which equals the `balanceOf` change of each non-zero endpoint when the two endpoints are distinct (a self-transfer leaves the wallet's balance unchanged), and every `TxShares` event reports multiplier `1e18` and shares equal to that raw value times `10 ** (18 - decimals)`

ID	Property
INV-73	`DSToken` never emits `TransferWithUIAmount`, in either mode

6.4.2 DSToken - SCALED_UI display and rounding

ID	Property
INV-13	For every raw `x`, `toUIAmount(x)` returns `floor(x * uiMultiplier() / 1e18)` evaluated at full precision, and reverts rather than returning a wrapped or truncated value when that result exceeds `type(uint256).max`
INV-14	For every UI amount `u`, `fromUIAmount(u)` returns `floor(u * 1e18 / uiMultiplier())` evaluated at full precision, and reverts rather than returning a wrapped or truncated value when that result exceeds `type(uint256).max`
INV-15	In every block, `balanceOfUI(a)` equals `toUIAmount(balanceOf(a))` and `totalSupplyUI()` equals `toUIAmount(totalSupply())`, both using the multiplier in force in that block, or each reverts exactly when the matching `toUIAmount` call reverts
INV-16	`fromUIAmount(toUIAmount(x)) <= x` for every raw `x` and every multiplier in force for which `toUIAmount(x)` returns a value
INV-17	The sum of `balanceOfUI(w)` over all wallets is at most `totalSupplyUI()` in every reachable SCALED_UI state in which `totalSupplyUI()` returns a value, for every token `decimals` value from 0 to 18
INV-18	For every holder `a`, `fromUIAmount` applied to `balanceOfUI(a)` is at most `balanceOf(a)`, and for every UI amount `u`, `toUIAmount` applied to `fromUIAmount(u)` is at most `u`, whenever these conversions return a value, so converting a displayed balance back never asks for more raw tokens than `a` holds and the raw amount a typed UI amount converts to never displays as more than that UI amount
INV-19	For a fixed multiplier in force, `toUIAmount` and `fromUIAmount` are non-decreasing in their argument over the arguments for which they return a value
INV-20	Whenever `uiMultiplier` returns `1e18`, `toUIAmount` and `fromUIAmount` return their argument unchanged, `balanceOfUI(a) == balanceOf(a)` for every `a`, and `totalSupplyUI() == totalSupply()`

6.4.3 TokenLibrary - UI multiplier schedule

ID	Property
INV-21	While SCALED_UI, the multiplier in force and `DSToken::newUIMultiplier()` are strictly positive 18-decimal fixed-point values in every reachable state, so `DSToken::fromUIAmount` never divides by zero
INV-22	Between two UI-multiplier administration calls, `DSToken::uiMultiplier()` changes at most once, at the first second with `block.timestamp >= DSToken::effectiveAt()` (inclusive), from the value in force when the schedule was set to `DSToken::newUIMultiplier()`, with no transaction required at the effective time
INV-23	Whenever `block.timestamp >= DSToken::effectiveAt()`, `DSToken::newUIMultiplier()` equals `DSToken::uiMultiplier()`; whenever `block.timestamp < effectiveAt()`, `newUIMultiplier()` is the value `uiMultiplier()` returns from `effectiveAt()` on unless the schedule is superseded or cancelled first

ID	Property
INV-24	`DSToken::setUIMultiplier(m, t)` succeeds only with `m > 0` and `t > block.timestamp`; afterwards `newUIMultiplier() == m`, `effectiveAt() == t`, and `uiMultiplier()` keeps the value in force before the call until `t`
INV-25	`DSToken::setUIMultiplier` while an update is pending discards that update without it ever taking effect and emits `UIMultiplierUpdateCancelled` before `UIMultiplierUpdated`; on a schedule that has already taken effect, or with nothing pending, it emits `UIMultiplierUpdated` only
INV-26	`DSToken::cancelUIMultiplierUpdate` succeeds only while `block.timestamp < effectiveAt()`, leaves `uiMultiplier()` at the value in force before the call for all later times until the next schedule, and sets `newUIMultiplier()` to that value and `effectiveAt()` to 0
INV-27	`UIMultiplierUpdateCancelled(m, t)` is emitted only when a schedule that has not yet taken effect is discarded (by supersede or cancel), and `m` and `t` equal the discarded `DSToken::newUIMultiplier()` and `DSToken::effectiveAt()`
INV-28	In every `UIMultiplierUpdated(old, new, t)` emitted by `DSToken::setUIMultiplier`, `old` equals `uiMultiplier()` at the moment of the call, `new` equals the stored `newUIMultiplier()` and `t` equals the stored `effectiveAt()` after the call
INV-29	Every change of the value `uiMultiplier()` returns is announced beforehand by a `UIMultiplierUpdated` event (from `DSToken::initialize`, an enable through `DSToken::setScaledUIAmountMode`, or `DSToken::setUIMultiplier`) naming the new value and its effective time, with no matching `UIMultiplierUpdateCancelled` in between
INV-30	Superseding a pending update with `DSToken::setUIMultiplier(m, t)` leaves the same `DSToken::uiMultiplier()` at every later time, the same `DSToken::newUIMultiplier()` and the same `DSToken::effectiveAt()` as `DSToken::cancelUIMultiplierUpdate` followed by `setUIMultiplier(m, t)` in the same block
INV-72	After a schedule set by `DSToken::setUIMultiplier(m, t)` takes effect, `DSToken::newUIMultiplier()` stays `m` and `DSToken::effectiveAt()` stays `t` (in the past) until the next `setUIMultiplier` call or mode switch

6.4.4 ERC-8056 and ERC-165 interface

ID	Property
INV-31	`DSToken::supportsInterface` returns true for the `IERC165` (`0x01ffc9a7`), `IERC20`, `IERC20Metadata` and `IERC20Permit` ids in both modes, and false for `0xffffffff`
INV-32	`DSToken::supportsInterface` returns true for each of `0xa60bf13d`, `0x4bd27648`, `0x57854fc3` and `0xd890fd71` exactly while SCALED_UI and false for each while REBASING, so its answer for these ids always equals `DSToken::scaledUIAmountEnabled`
INV-33	An integrator that applies `DSToken::uiMultiplier` to `balanceOf` whenever `DSToken::supportsInterface(0xa60bf13d)` returns true scales every balance exactly once: in every state where an ERC-8056 id is advertised and the token's `REBASING_PROVIDER` slot names a provider, `SecuritizeRebasingProvider::multiplier` of that provider is `1e18`, so `balanceOf` applies no rebasing factor, and the advertised functions answer without the `Scaled UI amount not enabled` revert
INV-34	`DSToken::supportsInterface` returns true only for interface ids whose functions the token implements in full and answers without the `Scaled UI amount not enabled` revert in its current mode

ID	Property
INV-35	While REBASING, `DSToken::uiMultiplier, newUIMultiplier, effectiveAt, toUIAmount, fromUIAmount, balanceOfUI, totalSupplyUI, setUIMultiplier, cancelUIMultiplierUpdate` all revert with `Scaled UI amount not enabled`
INV-36	`DSToken::initialize` leaves the token in SCALED_UI with `uiMultiplier() == newUIMultiplier() == 1e18` and nothing pending, and emits `UIMultiplierUpdated(0, 1e18, block.timestamp)`

6.4.5 Mode switching

ID	Property
INV-37	After initialization the mode changes only through a successful `DSToken::setScaledUIAmountMode`, and each such call flips `scaledUIAmountEnabled()` and emits `ScaledUIAmountModeChanged(enabled)` exactly once
INV-38	A successful `DSToken::setScaledUIAmountMode(true)` requires `SecuritizeRebasingProvider::multiplier` of the token's `REBASING_PROVIDER` to be exactly `1e18` when it executes, and leaves `DSToken::uiMultiplier()` and `DSToken::newUIMultiplier()` at `1e18` with nothing pending, discarding any UI schedule stored from an earlier SCALED_UI period, with `UIMultiplierUpdated(0, 1e18, block.timestamp)` emitted
INV-39	`DSToken::setScaledUIAmountMode(false)` succeeds only when `DSToken::uiMultiplier()` is `1e18` and no update is pending (`block.timestamp >= DSToken::effectiveAt()` and `DSToken::newUIMultiplier() == 1e18`)
INV-40	A mode switch changes no `balanceOf`, `totalSupply()` or share balance, and on the SCALED_UI side of the switch `DSToken::balanceOfUI(a) == balanceOf(a)` for every `a` and `DSToken::totalSupplyUI() == totalSupply()`, so no number an integrator reads changes meaning at the switch
INV-41	A MASTER `DSToken::setScaledUIAmountMode(false)` executing in a state with `DSToken::uiMultiplier() == 1e18`, nothing pending and provider multiplier `1e18` succeeds, and a MASTER `setScaledUIAmountMode(true)` executing on a REBASING token whose provider multiplier is `1e18` succeeds

6.4.6 SecuritizeRebasingProvider

ID	Property
INV-42	While a token is in SCALED_UI and has a rebasing provider wired, the provider at its `REBASING_PROVIDER` slot has `multiplier() == 1e18` in every reachable state, including after `setMultiplier`, a provider swap through `DSToken::setDSService`, a provider `DS_TOKEN` re-point, or a provider first wired after `DSToken::initialize`
INV-43	While a token is in SCALED_UI and `tokenData.totalIssued > 0`, its `REBASING_PROVIDER` provider has that token wired under `DS_TOKEN`, so the token's `REBASING_PROVIDER` slot and the provider's `DS_TOKEN` slot name each other and the provider-side freeze covers the token
INV-44	`setMultiplier` reverts while the token wired under the provider's `DS_TOKEN` reports SCALED_UI, otherwise changes `multiplier` only to a non-zero value, and emits `RebasingRateUpdated(old, new)` on every change

ID	Property
INV-45	While no token in SCALED_UI names a provider in its `REBASING_PROVIDER` slot, and either no token is wired under that provider's `DS_TOKEN` or the token wired there is in REBASING, MASTER can move that provider's multiplier through `setMultiplier` to any non-zero value, whichever DSToken implementation version is wired there (including one that predates `DSToken::scaledUIAmountEnabled`)
INV-46	`tokenDecimals` of a provider equals the `decimals` of every token whose `REBASING_PROVIDER` slot names that provider
INV-47	Every DS service that converts amounts between tokens and shares for a token (the token itself and its compliance service) converts through a provider with the same `multiplier` and `tokenDecimals` as the token's own `REBASING_PROVIDER`, so the token and compliance agree on every share amount in both modes

6.4.7 Mint throttle and over-cap mints

ID	Property
INV-48	Whenever `mintCapAmount > 0`, `mintedInWindow <= mintCapAmount`, `mintCapWindow > 0` and `overCapDelay > 0`
INV-49	With the cap active, the shares credited by throttled issuance (`DSToken::issueTokens`, `DSToken::issueTokensCustom`, `DSToken::issueTokensWithMultipleLocks`) since the current `windowStart` and before `windowStart + mintCapWindow` sum to at most `mintCapAmount` (shares), and an issuance that would exceed the remainder reverts in full
INV-50	For each throttled issuance, the share amount added to `mintedInWindow` (and emitted as `MintCapConsumed.amount`) equals the share amount credited to the recipient's wallet, so the cap is metered in the shares the mint creates, never in tokens or UI units
INV-51	Every issuance path other than `DSToken::executeOverCapMint` is metered by the cap, and `executeOverCapMint` never changes `mintedInWindow` or `windowStart`
INV-52	Every successful `DSToken::setMintCap` sets `windowStart` to `block.timestamp` and `mintedInWindow` to 0 and emits `MintCapUpdated`, and every throttled issuance with the cap active emits `MintCapConsumed(shares, mintedInWindow, windowStart)` with the post-issuance values
INV-77	Every successful `DSToken::setMintCap` stores the requested `mintCapAmount` and `mintCapWindow` and emits `MintCapUpdated` with them, and every successful `DSToken::setOverCapDelay`, `setOverCapGracePeriod` stores the requested value and emits `OverCapDelayUpdated` or `OverCapGracePeriodUpdated` with it
INV-53	With `mintCapAmount == 0`, throttled issuance never reverts for cap reasons and changes neither `windowStart` nor `mintedInWindow`
INV-54	While SCALED_UI, one throttled issuance of raw `v` to `to` and a sequence of throttled issuances to `to` summing to raw `v` within one window add the same shares to `mintedInWindow` and the same raw amount to `balanceOf(to)`
INV-74	With the cap active, the first throttled issuance at or after `windowStart + mintCapWindow` starts a new window: `windowStart` becomes that block's timestamp and `mintedInWindow` becomes that issuance's shares alone
INV-55	A given `(to, amount, salt)` is scheduled as an over-cap operation at most once ever: a re-submission reverts, including after that operation executed, was cancelled or expired

ID	Property
INV-56	An over-cap operation executes at most once (re-entrant calls included), never after cancellation, never before its `readyAt`, and never at or after a non-zero `expiresAt`; `executed` and `cancelled` are never both true and neither is ever cleared
INV-57	At scheduling, `readyAt` equals `block.timestamp + overCapDelay` and `expiresAt` equals `readyAt + overCapGracePeriod` when the grace period is non-zero and 0 (never expires) otherwise, and no later call (including `DSToken::setOverCapDelay, setOverCapGracePeriod`) changes the `to`, `amount`, `readyAt` or `expiresAt` of an existing operation
INV-58	A successful `DSToken::executeOverCapMint` credits exactly the scheduled `to` with the scheduled `amount` (token units, converted to shares at the provider multiplier in force at execution, i.e. raw `amount` in SCALED_UI), without caller-specified manual locks, and only after `validateIssuance` passes at execution time
INV-59	`DSToken::cancelOverCapMint` called by a `ROLE_ISSUER`, `ROLE_TRANSFER_AGENT` or `ROLE_MASTER` holder succeeds for any scheduled operation that is neither executed nor cancelled, at any time including after `readyAt` and after expiry
INV-60	`OverCapMintScheduled`, `OverCapMintExecuted` and `OverCapMintCancelled` are logged by the token address with the topic hashes and parameters declared in `IDSMintThrottle`, once per corresponding state change, and `OverCapMintScheduled` carries the stored `to`, `amount` and `readyAt`
INV-61	The token's over-cap behaviour equals its behaviour at `fb8a3b5` for every caller, input and state: `DSToken::scheduleOverCapIssuance, executeOverCapMint, cancelOverCapMint` succeed or revert in the same cases with the same reason strings, write the same `pendingMints` values, mint the same balances, return the same values and emit the same events in the same order

6.4.8 ServiceConsumer and access

ID	Property
INV-62	Only `ROLE_ISSUER` or `ROLE_MASTER` holders can change the UI multiplier schedule through `DSToken::setUIMultiplier` or `DSToken::cancelUIMultiplierUpdate`
INV-63	Only the `owner()` or a `ROLE_MASTER` holder can call `DSToken::setScaledUIAmountMode`, `SecuritizeRebasingProvider::setMultiplier`, `setDSService` (on the token, the provider and every other DS service), `DSToken::setMintCap, setOverCapDelay, setOverCapGracePeriod`
INV-64	Only `ROLE_ISSUER` or `ROLE_MASTER` holders can issue (`DSToken::issueTokensWithMultipleLocks` and its wrappers), schedule an over-cap mint or execute one, and only `ROLE_ISSUER`, `ROLE_TRANSFER_AGENT` or `ROLE_MASTER` holders can cancel one
INV-75	Only `ROLE_ISSUER`, `ROLE_TRANSFER_AGENT` or `ROLE_MASTER` holders can call `DSToken::burn`, and only `ROLE_TRANSFER_AGENT` or `ROLE_MASTER` holders can call `DSToken::seize`
INV-65	In every contract inheriting `ServiceConsumer`, each of `onlyMaster`, `onlyIssuerOrAbove`, `onlyIssuerOrTransferAgentOrAbove`, `onlyTransferAgentOrAbove` and `onlyExchangeOrAbove` admits exactly the callers it admitted at `fb8a3b5` (the `owner()` bypass applying to `onlyMaster` only) and, whenever the TrustService role lookup succeeds, rejects every other caller with `Insufficient trust level` before the function body runs

6.4.9 Storage and upgrade

ID	Property
INV-66	Upgrading a proxy deployed before this version preserves every pre-existing stored value: wallet and investor balances, `totalSupply` and `totalIssued` shares, allowances, the wallet list, `name`, `symbol`, `decimals`, `supportedFeatures`, `paused`, every mint-throttle parameter, and every `pendingMints` record read back field for field through `pendingMints(id)`
INV-67	The `TokenDataStore` storage footprint is unchanged by the upgrade (`scaledUIAmountMode` and `scaledUIAmount` occupy exactly the four slots removed from `__gap`), so no variable of `TokenDataStore` or of any contract laid out after it changes slot
INV-68	An upgraded proxy whose new storage is zero is in REBASING: `DSToken::scaledUIAmountEnabled()` is false, the ERC-8056 ids answer false, the ERC-8056 functions revert, and every pre-existing function returns what it returned before the upgrade
INV-69	The mode of a deployed proxy is set only by `DSToken::initialize` (once, at deployment) or by `owner()` or MASTER through `DSToken::setScaledUIAmountMode`, and its UI multiplier data only by those two and by `DSToken::setUIMultiplier, cancelUIMultiplierUpdate`, never by re-running an initializer on an upgraded proxy or by initializing the implementation contract
INV-70	Every state change and event of a `TokenLibrary` function called by the token takes effect in the token proxy's storage and is logged with the token proxy as emitter
INV-71	The DSToken implementation's runtime bytecode is under 24,576 bytes (EIP-170), and so is every library linked into it

Summary

Project Name	EVM DSToken ScaledUI Amount
Repository	dstoken
Commit	29690b113a8c...
Fix Commit	40a1fb54095f...
Audit Timeline	Oct 16th - Oct 19th, 2026
Methods	Manual Review

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	1	0	1
Informational	5	1	4
Gas Optimizations	0	0	0
Total	6	1	5

Summary of Findings

[L-1] `DSToken::setScaledUIAmountMode` enables SCALED_UI after a non-unit rebasing history, so for tokens with fewer than 18 decimals `balanceOf` stays nearest-rounded and `sum(balanceOfUI)` can exceed `totalSupplyUI`	Acknowledged
[I-1] `DSToken` never checks that its rebasing provider is frozen for it when SCALED_UI is enabled or the provider is re-pointed, so a later provider multiplier change rescales every raw balance	Acknowledged
[I-2] `SecuritizeRebasingProvider::setMultiplier` reverts whenever `DS_TOKEN` points at a token without `scaledUIAmountEnabled`, so wiring the provider before the token is upgraded blocks every rebase of a REBASING token	Acknowledged
[I-3] `TokenLibrary::setUIMultiplier` accepts any non-zero multiplier with no magnitude or ratio bound, so a mis-scaled value floors the ERC-8056 views to zero or makes them revert once it applies	Resolved
[I-4] Any ISSUER can make a queued master-timelock `DSToken::setScaledUIAmountMode(false)` revert at execution by leaving a UI multiplier update pending, even one that changes no displayed value	Acknowledged
[I-5] Pending over-cap mints scheduled in `SCALED_UI` mode keep their raw amount across a switch to Rebase mode, so a split re-applied through the rebasing provider before execution dilutes the recipient	Acknowledged

7 Findings

7.1 Low Risk

7.1.1 `DSToken::setScaledUIAmountMode` enables SCALED_UI after a non-unit rebasing history, so for tokens with fewer than 18 decimals `balanceOf` stays nearest-rounded and `sum(balanceOfUI)` can exceed `totalSupplyUI`

Description: The enable guard treats `getRebasingProvider().multiplier() == TokenLibrary.UI_MULTIPLIER_ONE` as proof that `balanceOf` is raw. `balanceOf` and `totalSupply` still convert shares with `RebasingLibrary::convertSharesToTokens`, which for `decimals < 18` rounds to nearest, so at `1e18` they equal the raw amount only when each share balance is a multiple of `10**(18 - decimals)`. Shares created while the token was REBASING at another multiplier generally are not. Nothing checks or rescales them, and the per-wallet balances and the total are rounded separately.

On a 6-decimal token at `1.25e18`, issuing 1 unit to each of three wallets credits `8e11` shares each. After MASTER sets the multiplier to `1e18` and enables SCALED_UI, the documented migration path, each wallet reads `DSToken::balanceOfUI` 1 while `totalSupplyUI()` reads 2. This breaks the documented `sum(balanceOfUI) <= totalSupplyUI()` and the raw-ERC-20 identity `sum(balanceOf) == totalSupply` that DeFi, bridges and custodians are told to rely on. The error is at most half a raw unit per wallet, so it grows with the holder count. Such a wallet also cannot transfer its full displayed balance, but that part follows any multiplier change and is the already-acknowledged round-to-nearest behaviour. Tokens with 18 decimals, or that only ever ran at `1e18`, are unaffected.

Proof of Concept: The test uses the real `deploy-all` task (6 decimals, REBASING, multiplier `1.25e18`, permissionless compliance with a stub registry) and the production entry points.

Save it as `test/securitize-io-pocs/ScaledUIRebasingHistoryRounding.test.ts`:

```
import { expect } from 'chai';
import { loadFixture } from '@nomicfoundation/hardhat-toolbox/network-helpers';
import hre from 'hardhat';
import { DSConstants } from '../../utils/globals';

const ONE = 10n ** 18n;
const REBASING_HISTORY = 1250000000000000000n; // 1.25e18

const deployRebasingSixDecimalsWithHistory = () =>
  hre.run('deploy-all', {
    name: 'Token Example 1',
    symbol: 'TX1',
    decimals: 6,
    multiplier: REBASING_HISTORY.toString(),
    compliance: 'PERMISSIONLESS',
    registryType: 'STUB',
    tokenMode: 'REBASING',
  });

describe('ScaledUIRebasingHistoryRounding', function () {
  it('PoC: ScaledUIRebasingHistoryRounding enabling scaled UI after a non-unit rebasing history leaves
  ↪  rounded balances and breaks the aggregate invariant', async function () {
    const { dsToken, trustService, rebasingProvider } = await
    ↪  loadFixture(deployRebasingSixDecimalsWithHistory);
    const [master, issuer, walletA, walletB, walletC, recipient] = await hre.ethers.getSigners();
    await trustService.connect(master).setRole(issuer, DSConstants.roles.ISSUER);

    // 1 raw unit each while the rebasing multiplier is 1.25e18 credits 8e11 shares
    for (const wallet of [walletA, walletB, walletC]) {
      await dsToken.connect(issuer).issueTokens(wallet, 1n);
    }
    expect(await rebasingProvider.convertTokensToShares(1n)).to.equal(800000000000n);

    // documented migration: move the multiplier to 1e18, then enable scaled UI
```

```
    await rebasingProvider.connect(master).setMultiplier(ONE);
    await dsToken.connect(master).setScaledUIAmountMode(true);

    const balances = [
      await dsToken.balanceOfUI(walletA),
      await dsToken.balanceOfUI(walletB),
      await dsToken.balanceOfUI(walletC),
    ];
    const totalUI = await dsToken.totalSupplyUI();

    // each 8e11 shares reads as 1 unit, the 2.4e12 total shares read as 2 units
    expect(balances).to.deep.equal([1n, 1n, 1n]);
    expect(totalUI).to.equal(2n);
    expect(balances[0] + balances[1] + balances[2]).to.be.greaterThan(totalUI);
    expect((await dsToken.balanceOf(walletA)) * 3n).to.be.greaterThan(await dsToken.totalSupply());

    // the displayed balance converts back to 1 unit = 1e12 shares, more than the 8e11 the wallet holds
    const raw = await dsToken.fromUIAmount(balances[0]);
    expect(raw).to.equal(1n);
    await expect(dsToken.connect(walletA).transfer(recipient, raw)).to.be.revertedWithoutReason();
  });
});
```

Run it with:

```
npx hardhat test test/securitize-io-pocs/ScaledUIRebasingHistoryRounding.test.ts
```

Recommended Mitigation: Enable SCALED_UI only when every wallet's share balance is a whole number of token units, the one condition under which `balanceOf` and `totalSupply` equal the raw amount and add up. Shares created at a multiplier of exactly `1e18` always are, so a token that never rebased, or has 18 decimals, passes unchanged; the check scans the wallet list the token already keeps and requires the scanned shares to sum to the total, which also catches a wallet whose rounded balance is zero and so is not listed. In `DSToken::setScaledUIAmountMode`:

```
            require(getRebasingProvider().multiplier() == TokenLibrary.UI_MULTIPLIER_ONE, "Rebasing
            ↪  multiplier must be 1e18");
+           TokenLibrary.requireWholeUnitShares(tokenData, walletsList, walletsCount, decimals);
            TokenLibrary.enableScaledUIAmount(scaledUIAmount);
```

and in `TokenLibrary`, after `TokenLibrary::enableScaledUIAmount`:

```
        emit UIMultiplierUpdated(0, UI_MULTIPLIER_ONE, block.timestamp);
    }
+
+   /// @dev SCALED_UI keeps balanceOf and totalSupply raw and additive only while every wallet's
↪  shares are whole token units
+   function requireWholeUnitShares(
+       TokenData storage _tokenData,
+       mapping(uint256 index => address wallet) storage _walletsList,
+       uint256 _walletsCount,
+       uint8 _decimals
+   ) public view {
+       uint256 unit = 10 ** (18 - _decimals);
+       if (unit == 1) return;
+       uint256 total;
+       for (uint256 i = 1; i <= _walletsCount; i++) {
+           uint256 shares = _tokenData.walletsBalances[_walletsList[i]];
+           require(shares % unit == 0, "Wallet shares not whole token units");
+           total += shares;
+       }
+       require(total == _tokenData.totalSupply, "Shares held outside the wallet list");
+   }
```

The loop costs a few thousand gas per listed wallet, so a token whose holder list is too long for one transaction (the effective execution cap on Arbitrum One included) cannot be switched in place, and the call should be simulated before it is queued through the master timelock. Because every operation at a multiplier of `1e18` moves whole units, a remainder left by an earlier multiplier persists while the multiplier stays at `1e18`, so a token whose rebasing history left one cannot be switched while it does. The check runs only at enable, so a token already switched with remainders keeps the reported state after the upgrade until MASTER switches it back to REBASING, which needs an identity UI multiplier.

The logic lives in `TokenLibrary` to stay inside the `DSToken` size limit. It ships as a `DSToken` implementation upgrade with a newly linked `TokenLibrary`. Once the check is in place, `test/securitize-io-pocs/ScaledUIRebasingHistoryRounding.test.ts` reverts at `setScaledUIAmountMode(true)` with `Wallet shares not whole token units`, so its remaining assertions must be replaced by an expectation of that revert; the enable bullet under "Switching the mode" in `docs/runbooks/scaled-ui-amount.md` section 2, FR-3 in `docs/dstoken-scaled-ui-amount-context.md` and the `IDSScaledUIAmount` NatSpec in `contracts/token/IScaledUIAmount.sol` should state the new enable condition.

Alternative mitigation: document an operator rule instead of scanning every wallet at enable. The proposed `requireWholeUnitShares` check works: with it applied, the PoC reverts at `setScaledUIAmountMode(true)` with `Wallet shares not whole token units`, and no other test changes. But it loops over every holder, which can exceed one transaction's gas on a large token, and a token with a rounding remainder can then never be enabled at `1e18`, because the remainder never goes away. A documentation change is consistent with how this rounding has been handled before: the knowledge base accepts nearest rounding for DSToken balances, and for the analogous independent-rounding effect of a non-unit multiplier on async vaults it records that "the mitigation is documented, not enforced" (knowledge base, "Vaults and asynchronous settlement"). Suggested addition to `docs/runbooks/scaled-ui-amount.md` section 2 ("Switching the mode") and section 3 (rounding), mirrored in the `IDSScaledUIAmount` NatSpec in `contracts/token/IScaledUIAmount.sol`, with section 6 of the context document updated to state the same rule:

```
Enabling after a rebasing history. Do not enable SCALED_UI on a token with fewer than 18 decimals whose
↪ shares were issued or transferred while the rebasing multiplier was not 1e18, even after the
↪ multiplier is back at exactly 1e18. Such a token usually holds share balances that are not whole
↪ token units, so balanceOf / balanceOfUI stay round-to-nearest and sum(balanceOfUI) <=
↪ totalSupplyUI() does not hold: each wallet can be off by up to half a raw unit in either direction.
↪ Before enabling a token that ever ran at another multiplier, check off-chain that every wallet's
↪ share balance is a multiple of 10^(18 - decimals) and that they add up to the total supply, and
↪ enable only if the check passes. Tokens with 18 decimals, and tokens that only ever ran at 1e18,
↪ need no check.
```

Securitize: Acknowledged. This requires shares created while the rebasing multiplier was not `1e18`. All Securitize tokens have only ever run at a provider multiplier of `1e18`, so every share balance is a whole multiple of `10^(18 - decimals)`, and `balanceOf` / `totalSupply` convert exactly. New tokens are deployed directly in `SCALED_UI` and never go through the `REBASING to SCALED_UI` switch.

7.2 Informational

7.2.1 `DSToken` never checks that its rebasing provider is frozen for it when SCALED_UI is enabled or the provider is re-pointed, so a later provider multiplier change rescales every raw balance

Description: In SCALED_UI mode `balanceOf` and `totalSupply` are meant to stay raw, which holds only while the rebasing provider's multiplier stays at `1e18`. The only guard on that is the freeze in `SecuritizeRebasingProvider::setMultiplier`, and it applies only when the provider's own `DS_TOKEN` service points back at the token. The token never checks this binding. `DSToken::setScaledUIAmountMode` checks only the multiplier when enabling, and `ServiceConsumer::setDSService` (`onlyMaster`, not `virtual`) replaces `REBASING_PROVIDER` with no check at all:

```
require(getRebasingProvider().multiplier() == TokenLibrary.UI_MULTIPLIER_ONE, "Rebasing multiplier must
↪  be 1e18");
```

A provider whose `DS_TOKEN` was never wired has no freeze, the state the provider's own comment records for "Providers with no token wired (legacy deployments)"; this change adds that wiring to `tasks/set-services.ts`. A provider proxy still on the old implementation has no freeze code at all. So when MASTER enables SCALED_UI on an upgraded legacy token whose provider is in either state, as the runbook's switching procedure describes, the call succeeds with the freeze inactive. If MASTER later calls `setMultiplier(2e18)`, it succeeds. That call is, for example, a corporate action run with the REBASING procedure, which operators expect the freeze to refuse given the runbook's "Frozen at 1e18 (the provider refuses to change it)". On a 2-decimal token, a holder's `balanceOf` then reads 2,000 instead of 1,000, and every pool, bridge and custodian that treats the token as a raw ERC-20 sees the rescaled amount. The token still advertises the four ERC-8056 ids, so if the same 2:1 split is also set through `DSToken::setUIMultiplier`, `DSToken::balanceOfUI` shows 4,000.

The harm needs a trusted MASTER to make that mistake, and recovery is another `setMultiplier`, which rescales every balance again. The documented legacy behaviour ("Providers with no token wired keep the previous behaviour") is correct for tokens that stay in REBASING. The threat model's assumption that the provider "is set once at deployment and never swapped" is not enforced on-chain.

Other impacts of the same root cause:

- `setDSService(REBASING_PROVIDER, newProvider)` on a SCALED_UI token rescales every balance at once when the new provider's multiplier is not `1e18` or its `SecuritizeRebasingProvider::tokenDecimals` differs (a provider at `1e18` with 2 decimals turns `1_000e6` raw on a 6-decimal token into `100_000`). A new provider without the back-pointer, including the first one wired after `DSToken::initialize` has already enabled the mode, is also unfrozen
- If the new provider is wired back and its multiplier is not `1e18`, the freeze pins that wrong value, and `setMultiplier(1e18)` reverts with "Multiplier is frozen in scaled UI mode"
- Each compliance service converts issuance and lock shares through its own `REBASING_PROVIDER` pointer, so after a one-sided rotation, locks are evaluated in a different unit from `balanceOf`

Recommended Mitigation: Have the token verify, when it enters SCALED_UI and on every `REBASING_PROVIDER` write while in that mode, that the provider reports itself frozen for this token, keeps `1e18` and uses the token's decimals. The provider answers from its own `DS_TOKEN` back-pointer and the token's flag, so a provider without the back-pointer, or still on the implementation without the freeze (which has no such function), fails the check. In `setDSService`, allow the override:

```
-    function setDSService(uint256 _serviceId, address _address) public override onlyMaster returns
↪  (bool) {
+    function setDSService(uint256 _serviceId, address _address) public virtual override onlyMaster
↪  returns (bool) {
```

In `DSToken`, override it and verify on entering the mode (the flag is set first because the provider reads it):

```
+    // while SCALED_UI is on, a new rebasing provider must be frozen at 1e18 for this token
+    function setDSService(uint256 _serviceId, address _address) public override returns (bool) {
+        super.setDSService(_serviceId, _address);
+        if (_serviceId == REBASING_PROVIDER && scaledUIAmountMode) {
```

```
+            TokenLibrary.requireFrozenRebasingProvider(getRebasingProvider(), decimals);
+        }
+        return true;
+    }
+
     /// @inheritdoc IDSScaledUIAmount
     function setScaledUIAmountMode(bool _enabled) external override onlyMaster {
         require(scaledUIAmountMode != _enabled, "Scaled UI amount mode unchanged");
         if (_enabled) {
-            // balanceOf must stay raw in this mode; the provider refuses to move away from 1e18
↪  afterwards.
-            require(getRebasingProvider().multiplier() == TokenLibrary.UI_MULTIPLIER_ONE, "Rebasing
↪  multiplier must be 1e18");
+            // the provider reads this flag when asked whether it is frozen, so set it before the check
+            scaledUIAmountMode = true;
+            TokenLibrary.requireFrozenRebasingProvider(getRebasingProvider(), decimals);
             TokenLibrary.enableScaledUIAmount(scaledUIAmount);
```

In `TokenLibrary`, a file-level interface and the check, where `address(this)` is the token because the library runs by delegatecall:

```
 import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";

+interface IFrozenRebasingProvider {
+    function tokenDecimals() external view returns (uint8);
+    function isMultiplierFrozenFor(address _token) external view returns (bool);
+}

 library TokenLibrary {
```

```
         emit UIMultiplierUpdated(0, UI_MULTIPLIER_ONE, block.timestamp);
     }
+
+    /// @dev SCALED_UI keeps balanceOf raw only while the provider is frozen at 1e18 for this token
+    function requireFrozenRebasingProvider(ISecuritizeRebasingProvider _provider, uint8 _decimals)
↪  public view {
+        require(_provider.multiplier() == UI_MULTIPLIER_ONE, "Rebasing multiplier must be 1e18");
+        require(IFrozenRebasingProvider(address(_provider)).tokenDecimals() == _decimals, "Rebasing
↪  provider decimals mismatch");
+        require(IFrozenRebasingProvider(address(_provider)).isMultiplierFrozenFor(address(this)),
↪  "Rebasing provider not frozen for this token");
+    }
```

In `SecuritizeRebasingProvider`, after `setMultiplier`:

```
         emit RebasingRateUpdated(old, _multiplier);
     }
+
+    /// @notice True when `_token` is wired under DS_TOKEN and reports SCALED_UI, so setMultiplier
↪  refuses to run
+    function isMultiplierFrozenFor(address _token) external view returns (bool) {
+        return getDSService(DS_TOKEN) == _token && IDSScaledUIAmount(_token).scaledUIAmountEnabled();
+    }
```

The code ships as a `DSToken` implementation upgrade with a newly linked `TokenLibrary`, plus a `SecuritizeRebasingProvider` proxy upgrade that must land first, and a one-word `ServiceConsumer` change that adds no bytecode to the other services. The checks live in `TokenLibrary` to keep `DSToken` under its size limit. The guard fixes an order for operators: the provider's `DS_TOKEN` service is wired to the token before the token's `REBASING_PROVIDER` service, and a new token meant for REBASING with a multiplier other than `1e18` is switched to REBASING (`setScaledUIAmountMode(false)`) before that wiring. Make that change in `tasks/set-services.ts` and `tasks/deploy-all.ts` (the `REBASING` fixtures in `test/utils/fixture.ts` and `test/dstoken-regulated.test.ts`

run through them) and in the external deployment service; compliance services keep their own REBASING_PROVIDER pointers, which must be re-pointed together with the token's.

Add tests for enabling and re-pointing against an unwired provider, an old-implementation provider, a non-1e18 provider and a decimals mismatch, and update the deploy order, the enable bullet and the freeze text in docs/runbooks/scaled-ui-amount.md section 2, FR-3 in docs/dstoken-scaled-ui-amount-context.md and the IDSScaledUIAmount NatSpec in contracts/token/IScaledUIAmount.sol. The checks run only on those two calls, so the upgrade leaves a token already in SCALED_UI as it is: for each such token, MASTER should wire the provider's DS_TOKEN to it and confirm that SecuritizeRebasingProvider::multiplier is 1e18 and tokenDecimals matches before relying on the freeze. A token whose provider multiplier has already moved off 1e18 is not repaired by the upgrade.

Alternative mitigation: extend the runbook with the safe procedure, instead of changing the code. The freeze works as designed when the token and its provider are wired the way set-services does it: the rebasing multiplier stays locked as long as the provider's DS_TOKEN entry points back at the token. The runbook's summary line, "Frozen at 1e18 (the provider refuses to change it)", and the matching IDSScaledUIAmount NatSpec leave out that this relies on the link, and section 2 mentions the link only briefly, so an operator upgrading an existing token or replacing a provider has no step-by-step guidance to keep it in place. We suggest adding that guidance to docs/runbooks/scaled-ui-amount.md (sections 1 and 2), with a short note in the NatSpec in contracts/token/IScaledUIAmount.sol, along these lines:

```
Keeping the rebasing multiplier locked in SCALED_UI mode. The provider refuses to change its multiplier
↪  while its DS_TOKEN entry points at this token. The contracts do not re-check this link after
↪  SCALED_UI mode has been turned on, so operators keep it in place by following three steps.

First, before turning SCALED_UI mode on, including on an existing token that was just upgraded, point
↪  the provider at the token with provider.setDSService(DS_TOKEN, token). After turning it on, confirm
↪  that both sides point at each other: the token's REBASING_PROVIDER entry is the provider, and the
↪  provider's DS_TOKEN entry is the token.

Second, only replace the token's rebasing provider (on the token itself or on any compliance service)
↪  with a provider whose multiplier is exactly 1e18, whose decimals match the token's decimals, and
↪  that already points back at the token. Replace it on the token and on every compliance service in
↪  the same timelock batch, so they never use different providers.

Third, never remove or change the provider's DS_TOKEN entry while the token is in SCALED_UI mode,
↪  because that silently removes the lock.
```

Securitize: Acknowledged. The rescaling requires a MASTER action: a setMultiplier call, or a provider swap or re-point while the token is in SCALED_UI. Securitize does not use rebasing and never calls setMultiplier. New tokens are deployed in SCALED_UI with the provider's DS_TOKEN wired back to the token, so the freeze is active from deployment. We do not plan to replace rebasing providers on live tokens.

7.2.2 SecuritizeRebasingProvider::setMultiplier reverts whenever DS_TOKEN points at a token without scaledUIAmountEnabled, so wiring the provider before the token is upgraded blocks every rebase of a REBASING token

Description: The new freeze in SecuritizeRebasingProvider::setMultiplier reads the token's mode through an unguarded external call on any non-zero DS_TOKEN:

```
require(!IDSScaledUIAmount(token).scaledUIAmountEnabled(), "Multiplier is frozen in scaled UI mode");
```

The pre-change DSToken has no DSToken::scaledUIAmountEnabled function and no fallback, and an address with no code returns nothing to decode, so both revert. ServiceConsumer::setDSService accepts any address. The runbook says the freeze "needs the token wired under service id DS_TOKEN (2) in the provider" but gives no upgrade order. A MASTER rolling this change out to an existing deployment may upgrade the provider and wire DS_TOKEN before upgrading the token proxy. From then on every setMultiplier call reverts. Upgraded proxies stay in REBASING mode, so for them setMultiplier is the normal entry point for a split or rate change to balanceOf. The block lasts until MASTER upgrades the token or calls setDSService(DS_TOKEN, address(0)) on the provider. After the governance handover, either step is a 48h master-timelock operation. No funds are lost.

Recommended Mitigation: Treat a wired token that has no code, or that reverts on `scaledUIAmountEnabled()`, as REBASING, and keep refusing when the token answers true. The code-size gate is needed because a typed call to an address without code fails while decoding the return value inside the caller, which `try` does not catch:

```
         address token = getDSService(DS_TOKEN);
-        if (token != address(0)) {
-            require(!IDSScaledUIAmount(token).scaledUIAmountEnabled(), "Multiplier is frozen in scaled
↪  UI mode");
-        }
+        // a token without code, or without scaledUIAmountEnabled, is a REBASING token
+        if (token.code.length > 0) {
+            try IDSScaledUIAmount(token).scaledUIAmountEnabled() returns (bool enabled) {
+                require(!enabled, "Multiplier is frozen in scaled UI mode");
+            } catch {}
+        }
```

A revert inside the `try` success block is not caught, so a token that answers true still blocks. A `DSToken` already on the implementation with `scaledUIAmountEnabled` cannot revert in that call, and the 63/64 gas rule leaves too little gas to finish `setMultiplier` after a starved call, so the `catch` never unfreezes it. The fix ships as a `SecuritizeRebasingProvider` proxy upgrade only. Add `test/rebasing-provider.test.ts` cases for `DS_TOKEN` set to an address without code and to a contract without the function (`setMultiplier` succeeds), and keep the existing "freezes the rebasing multiplier while enabled" test in `test/dstoken-scaled-ui-amount.test.ts` as the case that must still revert.

Alternative mitigation: document the upgrade order instead of changing the code. The revert only happens if the provider is wired to a token that has not been upgraded yet, so stating the order in the runbook is enough. Suggested addition to `docs/runbooks/scaled-ui-amount.md`:

```
Upgrade order for existing deployments. Upgrade the DSToken proxy to  4.3.0 before wiring the provider:
↪  only then call SecuritizeRebasingProvider::setDSService. The freeze in setMultiplier calls
↪  scaledUIAmountEnabled on the wired token, and a token on an older implementation does not have it,
↪  so wiring first makes every setMultiplier revert. If this happens, recover with
↪  provide::setDSService (or upgrade the token), then wire again.
```

Securitize: Acknowledged. The only effect is that `SecuritizeRebasingProvider::setMultiplier` reverts while the provider is wired to a token that has not been upgraded yet. Securitize does not use rebasing and never calls setMultiplier, so a temporary block of that function has no operational impact for us.

7.2.3 `TokenLibrary::setUIMultiplier` accepts any non-zero multiplier with no magnitude or ratio bound, so a mis-scaled value floors the ERC-8056 views to zero or makes them revert once it applies

Description: The only checks on a new UI multiplier are `_newMultiplier > 0` and `_effectiveAt > block.timestamp`. Every conversion then uses the stored value through `Math.mulDiv` against `TokenLibrary::UI_MULTIPLIER_ONE` (`1e18`). An ISSUER or MASTER typing `2` instead of `2e18` for a 2:1 split makes `DSToken::balanceOfUI` and `DSToken::totalSupplyUI` return 0 for a 6-decimal token whose holder balance and total supply are both 1,000,000 tokens, and `DSToken::fromUIAmount(1e6)` return `5e23` raw. A value near `type(uint256).max` makes `Math.mulDiv` overflow, so the views revert for any balance above one token of an 18-decimal token. With the documented one-second notice, `TokenLibrary::cancelUIMultiplierUpdate` cannot act once the value has applied, so the wrong value is live until a new schedule takes effect. Raw balances and transfers are unaffected. The harm falls on third-party ERC-8056 integrators, and the same role undoes it with another `DSToken::setUIMultiplier` call.

Recommended Mitigation: Replace the positivity check with a range check on the scheduled value, using constants that admit the documented split (`2e18`) and reverse split (`333333333333333333`) and reject a value typed without its 18 decimals or with them typed twice:

```
     uint256 internal constant UI_MULTIPLIER_ONE = 1e18;
+
+    /// @dev bounds on a scheduled multiplier, 1e-6 to 1e6 in 18-decimal fixed point
+    uint256 internal constant MIN_UI_MULTIPLIER = 1e12;
```

```
+    uint256 internal constant MAX_UI_MULTIPLIER = 1e24;
```

```
-        require(_newMultiplier > 0, "Multiplier must be positive");
+        require(_newMultiplier >= MIN_UI_MULTIPLIER && _newMultiplier <= MAX_UI_MULTIPLIER, "Multiplier
↪  out of range");
         require(_effectiveAt > block.timestamp, "Effective At must be in the future");
```

The bounds are library constants, so the change adds no `DSToken` bytecode. It ships as a new `TokenLibrary` deployment, a new `DSToken` implementation linked to it and a proxy upgrade; a multiplier already stored is not revalidated and is replaced by the next in-range `DSToken::setUIMultiplier` call. Update the "reverts for a zero multiplier" test in `test/dstoken-scaled-ui-amount.test.ts` to expect `Multiplier out of range`, add out-of-range cases (`2` and `1e25`), and change the `newMultiplier > 0` rule in `docs/runbooks/scaled-ui-amount.md` section 3 and the `m > 0` rule in `docs/dstoken-scaled-ui-amount-context.md` FR-5 to the range.

Securitize: Fixed in commitHash 40a1fb5.

Cyfrin: Verified. Implemented the recommended mitigation; The scheduled UI multiplier is now bounded between 1e12 and 1e24.

7.2.4 Any ISSUER can make a queued master-timelock `DSToken::setScaledUIAmountMode(false)` revert at execution by leaving a UI multiplier update pending, even one that changes no displayed value

Description: Leaving SCALED_UI requires the UI schedule to be at identity at the moment of execution:

```
block.timestamp >= scaledUIAmount.effectiveAt && scaledUIAmount.newMultiplier ==
↪  TokenLibrary.UI_MULTIPLIER_ONE,
```

`DSToken::setUIMultiplier` and `DSToken::cancelUIMultiplierUpdate` write that state instantly for any ISSUER. After handover, MASTER's disable is a queued operation with a 48h delay. An ISSUER can call `DSToken::setUIMultiplier(1e18, type(uint256).max)` before execution. The display does not change, yet the guard fails. The documented single call cannot clear that state itself, and `TokenLibrary::cancelUIMultiplierUpdate` reverts when nothing is pending, so a cancel-then-disable batch fails in the other case.

While the multiplier in force is `1e18`, only a batch that schedules and then cancels an update, `DSToken::setUIMultiplier(1e18, type(uint256).max)`, `DSToken::cancelUIMultiplierUpdate()` and the disable, executes whether or not an update is pending, and the disable rule in `docs/runbooks/scaled-ui-amount.md` section 2 names no such batch. An honest ISSUER scheduling a routine corporate action during the queue window wastes the cycle in the same way. While the token stays in SCALED_UI, a provider whose `DS_TOKEN` points back at it keeps `SecuritizeRebasingProvider::setMultiplier` frozen. The harm is a bounded delay of a MASTER decision; no funds are at risk.

Recommended Mitigation: Make the disable check read the multiplier in force instead of the ISSUER-writable schedule, and let the MASTER call drop an update that is still pending. A pending update has not yet changed any balance an integrator displays, so only an in-force non-identity multiplier, which leaving the mode would change, still blocks. In `DSToken::setScaledUIAmountMode`:

```
             // Leaving the mode must not change any number an integrator reads: the display multiplier
-            // has to be identity with nothing scheduled. (The rebasing multiplier is 1e18 already.)
-            require(
-                block.timestamp >= scaledUIAmount.effectiveAt && scaledUIAmount.newMultiplier ==
↪  TokenLibrary.UI_MULTIPLIER_ONE,
-                "UI multiplier must be 1e18 with no pending update"
-            );
+            // in force has to be identity, and a pending update is dropped instead of blocking the
↪  switch
+            TokenLibrary.disableScaledUIAmount(scaledUIAmount);
```

and after `TokenLibrary::enableScaledUIAmount`, so the logic stays out of `DSToken`:

```
         emit UIMultiplierUpdated(0, UI_MULTIPLIER_ONE, block.timestamp);
```

```
    }
+
+    /// @dev Leaving SCALED_UI: the multiplier in force must be identity; a pending update is announced
↪  as cancelled and cleared
+    function disableScaledUIAmount(ScaledUIAmountData storage _data) public {
+        require(currentUIMultiplier(_data) == UI_MULTIPLIER_ONE, "UI multiplier must be 1e18");
+        if (block.timestamp < _data.effectiveAt) {
+            emit UIMultiplierUpdateCancelled(_data.newMultiplier, _data.effectiveAt);
+        }
+        _data.multiplier = UI_MULTIPLIER_ONE;
+        _data.newMultiplier = UI_MULTIPLIER_ONE;
+        _data.effectiveAt = 0;
+    }
```

It ships as a `DSToken` implementation upgrade with a newly linked `TokenLibrary`, and the call replaces inline code, so `DSToken` is not expected to grow (the size test confirms it). Update the two "disabling reverts" tests in `test/dstoken-scaled-ui-amount.test.ts` (a pending update with an identity multiplier in force now succeeds and emits `UIMultiplierUpdateCancelled`; the revert string becomes `UI multiplier must be 1e18`), the disable rule in `docs/runbooks/scaled-ui-amount.md` section 2 and `docs/dstoken-scaled-ui-amount-context.md` FR-3, and the `IDSScaledUIAmount` NatSpec in `contracts/token/IScaledUIAmount.sol`.

- With the fix, the disable also silently cancels a pending non-identity update (for example a split scheduled for next week), with `UIMultiplierUpdateCancelled` emitted. The runbook should say that disabling cancels any scheduled corporate action.
- It does not cover a token that has to unwind a split before the switch. With `2e18` in force and a legitimate `setUIMultiplier(1e18, effectiveAt)` pending, an ISSUER that supersedes it with `setUIMultiplier(1e18, type(uint256).max)` keeps `2e18` in force indefinitely, so the disable keeps reverting with `UI multiplier must be 1e18`
  - For this case, an operational fallback works with the current code. If the ISSUER keeps superseding or cancelling the scheduled update, MASTER removes the ISSUER role (`TrustService::removeRole` through the timelock) from every ISSUER account before it schedules the unwind. MASTER then schedules `setUIMultiplier(1e18, effectiveAt)` itself and queues the disable to execute after that `effectiveAt`. Once the switch has executed, it grants the ISSUER role again. With no ISSUER left, only MASTER can call `DSToken::setUIMultiplier` or `DSToken::cancelUIMultiplierUpdate` (both are `onlyIssuerOrAbove`), so the scheduled update is guaranteed to take effect. The cost is at least one extra 48h cycle, during which the token has no issuer for issuance or corporate actions. A code-level alternative is to let a schedule set by MASTER be changed only by MASTER. That closes both cases without removing roles, but it is a design change for the client to decide.

Securitize: Acknowledged. This only affects disabling `SCALED_UI` (`setScaledUIAmountMode(false)`). Securitize does not use rebasing and does not plan to switch any token back to `REBASING`, so the disable path is not part of our operations. If it is ever needed, the operator will coordinate with ISSUERs so that no UI multiplier update is scheduled during the timelock window, or will use the schedule-then-cancel batch described in the finding. `ISSUER` is a trusted, permissioned role, and the impact is limited to a delayed `MASTER` action with no funds at risk.

7.2.5 Pending over-cap mints scheduled in `SCALED_UI` mode keep their raw amount across a switch to Rebase mode, so a split re-applied through the rebasing provider before execution dilutes the recipient

Description: The over-cap timelock stores the mint amount exactly as the issuer passed it, and converts it to shares only when the mint executes. `TokenLibrary::scheduleOverCapIssuance` records `amount: _amount`, and `DSToken::executeOverCapMint` hands that number to `DSToken::_issueUncapped`. The conversion to shares happens in `TokenLibrary::issueTokensCustom` at the rebasing multiplier in force at execution time.

In `SCALED_UI` mode the stored amount is raw, so it is in pre-corporate-action units. A split or consolidation set through `DSToken::setUIMultiplier` while the mint is pending therefore applies to it exactly as it applies to existing holders. `DSToken::setScaledUIAmountMode(false)`, which switches a token back to REBASING only once the UI multiplier is `1e18`. For a token that already split on the display layer, keeping that split in REBASING means the

operators have to unwind it with `DSToken::setUIMultiplier` back to `1e18`, disable SCALED_UI, then re-apply it with `SecuritizeRebasingProvider::setMultiplier`.

- **After the switch**, the pending amount is read as post-split tokens. The re-applied split doubles every existing holder's `balanceOf`, while the pending mint converts at the new rate into proportionally fewer shares. The recipient ends up with a smaller share of the token than they were entitled to when the mint was scheduled.

Impact: The recipient of a pending over-cap mint ends up with a smaller share of the token than they were entitled to when the mint was scheduled, reduced by the ratio of the split re-applied through the provider.

- For example, a 2:1 split halves it, and the existing holders gain the difference. The mint executes without any error, so the shortfall shows up only when someone reconciles the holder's position against the issuance record. Correcting it after execution needs another issuance, which counts against the mint cap again.

No funds can be taken by a third party, and the outcome can be avoided by cancelling and rescheduling pending mints before the switch.

Proof of Concept: A DSToken of 6 decimals, SCALED_UI. Bob holds 100 stocks issued before a 2:1 split on the display layer. The issuer then schedules an over-cap mint of 100 stocks for Alice, i.e. `fromUIAmount(100 stocks)` = 50,000,000 raw. At that point Alice's promised position is half of Bob's displayed 200 stocks. The token then follows the switch sequence (UI multiplier back to `1e18`, switch to rebase mode, provider multiplier `2e18`), and the mint executes once the delay has passed:

	Alice	Bob	Alice / Bob
At scheduling (SCALED_UI, UI 2e18)	100 stocks promised (50,000,000 raw stored)	200 shown	0.5
After execution (REBASING, provider 2e18)	50 stocks	200 stocks	**0.25**

Recommended Mitigation: Document in `docs/runbooks/scaled-ui-amount.md`, in the mode-switch section and next to `SecuritizeRebasingProvider::setMultiplier`, that pending over-cap mints must be cancelled with `DSToken::cancelOverCapMint` before the token is switched to REBASING or rebased. They should then be rescheduled with amounts expressed at the new rate. A rescheduled mint needs a new amount or salt, and it restarts the over-cap delay.

Securitize: Acknowledged. This only applies when a token is switched from `SCALED_UI` back to `REBASING` and then rebased through the provider while an over-cap mint is pending. Securitize does not use rebasing: existing tokens have always run at a provider multiplier of `1e18`, new tokens are deployed in `SCALED_UI`, and corporate actions will be applied only through `setUIMultiplier`. We do not plan to switch tokens back to `REBASING`, so this sequence does not occur in our deployments.